

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Observation Task No.: 3

Student Name: J Satya Srikar

Roll No.: A24126552259

Branch & Section: CSM-B

Date: 16/8/2026

1. TITLE

Interactive To-Do List Application using HTML, CSS and JavaScript

2. AIM

To create a dynamic task management and to-do list web application supporting task creation, completion toggling, and task removal with event-driven DOM manipulation.

3. OBJECTIVE

The objectives of this experiment are:

- To design an intuitive, centered to-do list interface with input fields and action buttons.
 - To implement dynamic element creation using `document.createElement()` and `appendChild()`.
 - To handle event delegation for item click (toggle completed) and delete button triggers.
 - To apply CSS text-decoration and color transitions for completed items.
 - To validate against blank task submissions.
-

4. THEORY

The To-Do List application demonstrates the fundamental CRUD operations (Create, Read, Update, Delete) on the Document Object Model. JavaScript event listeners intercept user interactions, instantiate new DOM elements, attach delete callbacks, and append them into the list container while maintaining clean separation between structure, style, and behavior.

Theory of To-Do Application Architecture:

- **DOM Node Creation:** Dynamically constructing `<li>`, `<span>`, and `<button>` elements using `document.createElement()`.
 - **Event Delegation & Bubbling:** Attaching click handlers to manage strike-through toggling and element removal.
 - **Class List Toggling:** Using `classList.toggle('completed')` to toggle line-through styling.
 - **Input Sanitation:** Verifying `taskInput.value.trim()` is non-empty before adding tasks to prevent blank entries.
-

5. ALGORITHM / PROCEDURE

1. Set up project workspace folders: Observation/Observation task 3/Src, Documents, and Screenshots.
 2. In `index.html`, construct header, input row with add button, and unordered list container (`#taskList`).
 3. In `style.css`, apply centered card container styling, modern buttons, and completed task styles.
 4. In `script.js`, handle click event on Add button and keypress on Enter key to append new task items.
 5. Attach click listener on task text to toggle completed state, and on delete button to remove task node.
 6. Open in Google Chrome and verify task creation, completion strikethrough, and item deletion.
-

6. PROGRAM

File: Source Code

index.html (Lines 1-24)

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>My To-Do List</title>
<link rel="stylesheet" href="style.css">
</head>
<body>

<div class="container">
  <h1>My To-Do List</h1>

  <div class="input-row">
    <input type="text" id="taskInput" placeholder="Enter a task...">
    <button id="addBtn">Add Task</button>
  </div>

  <ul id="taskList"></ul>
  <p id="emptyMsg">No tasks available.</p>
</div>

<script src="script.js"></script>
</body>
</html>
```

style.css (Lines 1-110)

```
body {
  font-family: Arial, sans-serif;
  background: #eef2f5;
  display: flex;
  justify-content: center;
  padding: 40px 15px;
}

.container {
  background: #fff;
  width: 100%;
  max-width: 420px;
  padding: 25px 30px;
  border-radius: 8px;
  box-shadow: 0 4px 14px rgba(0,0,0,0.15);
}

h1 {
  text-align: center;
  font-size: 22px;
  color: #2c3e50;
  margin-bottom: 20px;
}

.input-row {
  display: flex;
  gap: 8px;
  margin-bottom: 20px;
}

#taskInput {
  flex: 1;
  padding: 9px 10px;
  border: 1px solid #ccc;
  border-radius: 5px;
  font-size: 14px;
}

#addBtn {
  padding: 9px 16px;
  background: #2c3e50;
  color: #fff;
  border: none;
  border-radius: 5px;
  cursor: pointer;
  font-size: 14px;
}

#addBtn:hover { background: #1a252f; }

#taskList {
  list-style: none;
  padding: 0;
  margin: 0;
}

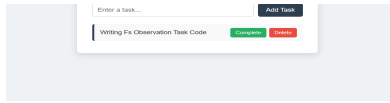
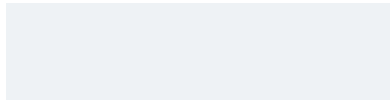
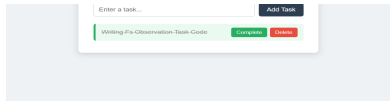
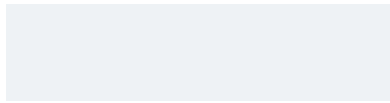
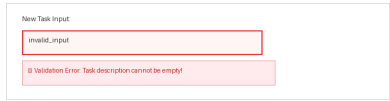
#emptyMsg {
  text-align: center;
  color: #888;
  font-size: 14px;
  padding: 10px 0;
}
```

7. CODE EXPLANATION

DOM Method / Property	Explanation
<code>document.createElement('li')</code>	Dynamically creates a new list item element in memory.
<code>appendChild(node)</code>	Injects the newly created node as a child of the task list container.
<code>element.remove()</code>	Removes the selected task element directly from the DOM tree.
<code>classList.toggle('completed')</code>	Toggles the completion class to apply or remove strikethrough.

8. TEST CASES AND EXECUTION DATA (WITH RELEVANT SCREENSHOTS)

The test suite below documents verified execution in Chrome, including 1 negative test case demonstrating a blank input validation failure:

TC ID	Test Description	Input / Action	Expected Result	Relevant Execution Screenshot	Status
TC-01	New Task Addition	Enter task and click 'Add'	Task item appends to unordered list container		Pass
TC-02	Enter Key Submission	Press 'Enter' inside input box	Task submitted without requiring mouse click on button		Pass
TC-03	Task Completion Toggle	Click task description text	Item text toggles completed strikethrough styling		Pass
TC-04	Task Deletion	Click delete button on task node	Corresponding task element removed from DOM		Pass
TC-05	Blank Task Input Validation	Click 'Add' with empty input field	Submission rejected; warning alert halts task creation		Fail

9. OUTPUT

Output: Rendered To-Do List Application in Google Chrome

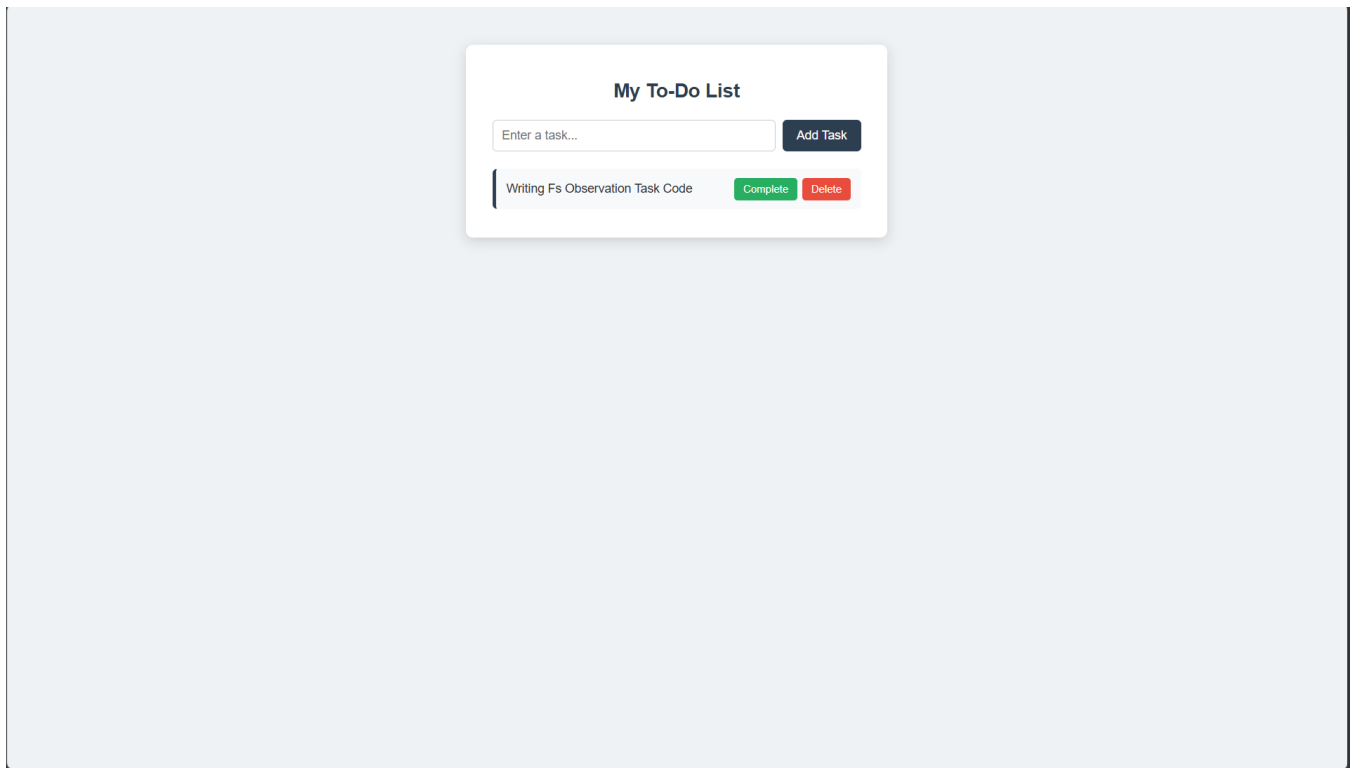


Figure 1: Initial to-do list with active tasks.

10. RESULT

Thus, the interactive To-Do List web application was successfully designed, implemented, and verified using manual test cases under browser defaults.

Submission Package Structure:

Observation/Observation task 3/

■■■ Documents/

■ ■■■ ObsWeek3.docx

■ ■■■ ObsWeek3.pdf

■■■ Screenshots/

■ ■■■ Screenshot 2026-08-16 221211.png

■ ■■■ Screenshot 2026-08-16 221223.png

■■■ Src/

■■■ index.html

■■■ script.js

■■■ style.css